

Team Tracks & Task Checklists (M1–M5)

How to use this: each teammate picks the track that fits them best (niche hints below). Within a track, do the tasks **in the order listed**. Tasks marked ★ **unblock other people**, so finish them first and post in the group when they're done. Every task names its dependencies. "—" means you can start right away.

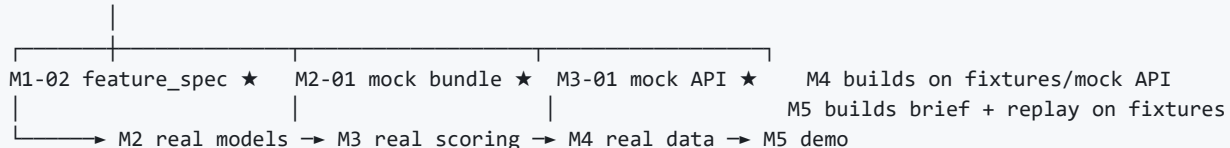
Each track has a GitHub issue with the same checklist (label `track:M1` ... `track:M5`). Tick boxes there as you go.

Pick your track

Track	Name	Pick this if you like...	Owns
M1	Data & Features	pandas, data cleaning, statistics	<code>ml/data/</code> , <code>nscore/features/</code> , <code>nscore/drift/</code>
M2	ML Modeling & MLOps	scikit-learn, experiments, Azure ML	<code>ml/train/</code> <code>evaluate/</code> <code>explain/</code> <code>registry/</code> , <code>nscore/detection/</code> , <code>nscore/bundle/</code>
M3	Backend & Platform	APIs, databases, DevOps	<code>api/</code> , <code>infra/</code> , <code>.github/workflows/</code>
M4	SOC Console (Frontend)	UI/UX, visualisation	<code>dashboard/</code>
M5	Security, GenAI & Demo	security, LLM prompting, storytelling	<code>api/app/services/brief.py</code> , <code>replay/</code> , <code>nscore/contracts/policy.py</code> <code>values</code> , the deck

Why the tracks can run in parallel

Day 0 (already done in repo): `contracts/schemas.py` · `policy.py` · `fusion.py` · `fixtures/*.json` · `tests`



Only **four** hard hand-offs exist: `feature_spec.json` (M1→all), mock bundle (M2→M3), mock API (M3→M4), real bundle v1 (M2→M3). Everything else is built against the contracts and fixtures.

Integration milestones (everyone)

Milestone	Exit criteria	Owners
I0 Foundation	<code>feature_spec.json</code> merged · mock bundle loads in the API · mock API serves all endpoints · dashboard renders every page from mock data · CI green	M1, M2, M3, M4
I1 First real model	bundle v1 (real data) loaded from a local ref · <code>replay</code> → <code>/v1/flows</code> → incident appears in the dashboard	all
I2 Feature complete	Azure ML pull + cache fallback · briefs (LLM + template) · drift live · holdout-botnet bundle · LOAO chart in the UI	all
I3 Demo ready	2 timed dry runs · recorded fallback video · model card + README final · deck claims checked against what's built	all

Schedule I3 at least **2–3 days before Round 2**, not the night before.

M1 — Data & Features

- ☐ **M1-01** Download the **corrected CICIDS2017** (Engelen et al., distrinet-research.be). Fall back to the original `MachineLearningCSV` if it's unavailable. Record file checksums and row counts per label in `data/README.md`.
Deps: —
- ☐ **M1-02** ★ Write the canonical `nscore/contracts/feature_spec.json`: raw → snake_case names, dtypes, clip ranges, log1p flags, and the dropped columns with reasons (identifier or leakage). Run correlation pruning ($|p| > 0.95$). Open a PR; M2 and M3 review it. *Deps:* M1-01 → **unblocks M2, M3**
- ☐ **M1-03** `ml/data/clean.py`: Inf/NaN report and handling (counts logged, nothing silently zero-filled), dedupe, label → `AttackFamily` mapping (Rare bucket, "Attempted" → BENIGN), output parquet to `data/processed/`.
Deps: M1-01
- ☐ **M1-04** EDA notebook + `docs/data_profile.md`: class balance per day, missing values, top correlated pairs, what got dropped and why. *Deps:* M1-03
- ☐ **M1-05** `ml/data/split.py`: **time-blocked split per (day, label), 70/15/15**, saved as index files plus `split_manifest.json`. A LOAO fold generator (one fold per family). Unit test: no flow index appears in two splits, and every family appears in train/val/test. *Deps:* M1-03 → **unblocks M2-02**
- ☐ **M1-06** `nscore/features/transform.py` (`FeatureSpec`, `FlowTransformer`): fit on train only, `transform(df)` and `transform_records(list[dict])` share one code path. **Parity test:** the same flows through both paths give identical arrays. *Deps:* M1-02
- ☐ **M1-07** `nscore/drift/psi.py`: `build_reference()` (quantile bins of the top-K features) and `psi()` (epsilon-smoothed), with unit tests on synthetic shifted distributions. *Deps:* M1-06
- ☐ **M1-08** `ml/data/baseline_stats.py`: benign median and p95 per feature → `baseline_stats.json` (powers the "vs normal" explanations). *Deps:* M1-05
- ☐ **M1-09** Replay extracts for M5: test-split slices per scenario (benign background, brute-force burst, DDoS, botnet) as CSVs with metadata and ground truth, ≤ 10k flows each, in `data/replay/`. *Deps:* M1-05
- ☐ **M1-10** Data section of `docs/model_card.md`: dataset version, cleaning counts, known dataset issues (Engelen et al.), split rationale. *Deps:* M1-04, M1-05

M2 — ML Modeling & MLOps

- ☐ **M2-01** ★ `scripts/make_mock_bundle.py`: a tiny bundle (RF + IF trained on random data with the feature names from `feature_spec.json`, or the fixture feature names if the spec isn't merged yet), with a valid `manifest.json` and sha256s. *Deps:* — → **unblocks M3-03**
- ☐ **M2-02** `ml/evaluate/metrics.py`: an evaluation harness producing an `EvaluationReport` (per-class P/R/F1/**FPR**/AUC, macro-F1, binary ROC/PR-AUC, confusion matrix PNG + JSON). Then a **baseline** `rf_binary` on the P1 split. *Deps:* M1-05, M1-06
- ☐ **M2-03** Imbalance study: `class_weight="balanced_subsample"` vs SMOTE (train only, inside the CV folds via an `imblearn` Pipeline, on down-sampled benign). Before/after table in `docs/experiments.md`. Pick one. *Deps:* M2-02
- ☐ **M2-04** Hyperparameter search (`RandomizedSearchCV` with time-blocked / grouped CV, not plain k-fold). Set a time budget and log it. *Deps:* M2-03

- ☐ **M2-05** `rf_multiclass` on attack flows only, with the Rare bucket. Per-class report. *Deps: M2-02*
- ☐ **M2-06** `iforest` trained on **benign train flows only**. Save the benign val score distribution for percentile mapping. *Deps: M1-06*
- ☐ **M2-07** Threshold calibration: `tau_binary` at benign FPR $\leq 1\%$ on val, `tau_anomaly` = p99.5 of benign val. Reliability plot. Write `thresholds.json`. Check `nscore/detection/fusion.py` against real numbers. *Deps: M2-04, M2-06*
- ☐ **M2-08 ★ Headline result: LOAO experiment.** For each of the 6 families, retrain without it and record recall on it for RF-only vs fusion, at the calibrated FPR. Outputs `loao` entries in the report plus `loao.png`. *Deps: M2-07, M1-05*
- ☐ **M2-09** SHAP: `TreeExplainer` for `rf_binary` and `iforest`, a top-k local explanation function (raw value + shap + benign median), global importance, and a latency benchmark (< 50 ms/flow target). *Deps: M2-07, M1-08*
- ☐ **M2-10** `nscore/bundle/`: packager (manifest + sha256) and `load_bundle("local:...")` with hash verification. Build **bundle v1 (full)** and **demo-holdout-botnet**. *Deps: M2-08, M2-09, M1-07* → **unblocks M3 real scoring (I1)**
- ☐ **M2-11** Azure ML: create the workspace, register both bundles with tags, implement `load_bundle("azureml:...")` with a cache fallback, and **prove it works by pulling into a clean environment**. *Deps: M2-10*
- ☐ **M2-12** Model card (`docs/model_card.md`) metrics, LOAO, operating point, **drift section**, limitations. Done with M1 (data section) and M5 (Q&A wording). *Deps: M2-08*

M3 — Backend & Platform

- ☐ **M3-01 ★** FastAPI skeleton with **mock mode** (`NS MOCK=1`): every endpoint in 03 §4.1 returns the matching fixture, typed with the contract models. `/docs` works. *Deps: —* → **unblocks M4**
- ☐ **M3-02** SQLite schema (03 §4.4), WAL mode, repository layer, `scripts/init_db.py`. *Deps: —*
- ☐ **M3-03** Load the bundle at startup via `load_bundle(MODEL_REF)` (mock bundle until v1 exists). Real `/health`, `/v1/model`, `/v1/model/evaluation`. *Deps: M2-01*
- ☐ **M3-04** `POST /v1/flows`: validate → `FlowTransformer` → RF + IF → `fuse` → SHAP for non-benign → persist. Batches of up to 500. Record latency per flow. *Deps: M3-02, M3-03, M1-06*
- ☐ **M3-05** Incident correlator (03 §4.3) with unit tests: windowing, key separation, no merging into resolved or dismissed incidents. *Deps: M3-04*
- ☐ **M3-06** Priority and MITRE enrichment via `nscore.contracts.policy`. *Deps: M3-05*
- ☐ **M3-07** `GET /v1/incidents` (filter/sort/paginate), `GET /v1/incidents/{id}`, `POST /v1/incidents/{id}/actions` with an audit row and status transitions (stamp `acknowledged_at` for MTTA). *Deps: M3-05*
- ☐ **M3-08** Drift monitor service (rolling window → `nscore.drift.psi`, snapshots) and `/v1/drift`. `/v1/metrics` (throughput, p50/p95, confirmed precision, FP rate, MTTA). *Deps: M3-04, M1-07*
- ☐ **M3-09** Wire in M5's brief service: `GET /v1/incidents/{id}/brief` (lazy, cached, timeout → template). `POST /v1/admin/reload-model`. *Deps: M3-07, M5-03*
- ☐ **M3-10** Hardening: API key, `x-Analyst`, admin key, structured JSON logs with request ID, 422 messages that name missing features, CORS. *Deps: M3-07*

- ☐ **M3-11** Tests (`api/tests/`): every endpoint, happy path and errors (malformed flow, missing model), against the mock bundle. Make the CI checks required. `infra/docker-compose.yml` (api + dashboard, one command). *Deps: M3-07*
- ☐ **M3-12** (*stretch*) Deploy the API and dashboard to Azure Container Apps. *Deps: M3-11*

M4 — SOC Console

- ☐ **M4-01** ★ Streamlit skeleton + `dashboard/api_client.py` typed with the contract models. Two data sources: `NS_API_URL` (live or mock API) and an **offline mode** that reads the fixtures directly. You can start this today. *Deps: —*
- ☐ **M4-02** Visual system: P1–P4 colours, `Known attack / Novel anomaly` chips, status pills, dark SOC theme, `dashboard/theme.py`. *Deps: M4-01*
- ☐ **M4-03 Live Queue** page: sorted by priority, filters, header counters, 3 s auto-refresh (`st.fragment(run_every=...)`), new-row highlight, pagination for 500+ incidents. *Deps: M4-02*
- ☐ **M4-04 Incident Detail**: SHAP bar chart, **"vs normal" table**, MITRE badge linking to `attack.mitre.org`, metadata, brief panel (loading / LLM / template label), action buttons with a note, action timeline. *Deps: M4-03*
- ☐ **M4-05** Analyst sign-in (name + role) in session state, sent as `X-Analyst`. *Deps: M4-04*
- ☐ **M4-06 Model & Evaluation** page: version and registry ref, per-class table, confusion-matrix heatmap, **LOAO grouped bar chart** (RF-only vs fusion), operating FPR, limitations. *Deps: M4-02*
- ☐ **M4-07 Drift & Health** page: PSI bars with 0.10/0.25 reference lines, status banner, throughput and latency tiles. *Deps: M4-02*
- ☐ **M4-08 Analyst Metrics** page: confirmed precision, FP dismiss rate, MTTA, actions per analyst. *Deps: M4-02*
- ☐ **M4-09** Switch from the mock API to the real one at I1, and fix any contract gaps (raise a contract PR, don't hack around it). *Deps: M3-07*
- ☐ **M4-10** Polish: empty, error and loading states, API-down banner, a screenshot set for the deck. *Deps: M4-09*

M5 — Security, GenAI & Demo

- ☐ **M5-01** Threat research: validate or adjust `MITRE_MAP` and `SEVERITY` in `nscore/contracts/policy.py` and justify them in `docs/threat_model.md`. Write one "suggested next step" playbook line per family (used by the template brief). *Deps: —*
- ☐ **M5-02** Provision Azure OpenAI and a gpt-4o-mini class deployment. Fill in `.env.example` keys and document the setup in `docs/azure_setup.md` (shared with M2's Azure ML notes). *Deps: —*
- ☐ **M5-03** ★ `api/app/services/brief.py`: `generate_brief(incident: IncidentDetail) -> Brief`. Grounded prompt (03 §4.6), confidence-band hedging, novel-anomaly wording, 8 s timeout, **deterministic template fallback**. Unit tests with a stubbed client. Built against `fixtures/incident_detail.json`, so it doesn't need the API. *Deps: M5-01, M5-02 → unblocks M3-09*
- ☐ **M5-04** Brief evaluation: 10 varied incidents (each family, low and high confidence, novel). Check for invented facts and correct hedging. Results table in `docs/brief_eval.md`. *Deps: M5-03*
- ☐ **M5-05** `replay/replay.py`: reads replay CSVs → batches → `POST /v1/flows` with pacing (`--speed`), scenario files `replay/scenarios/*.yaml` (background + bursts), and live ground-truth scoring printed to the console. Built against the mock API first. *Deps: M3-01*

- ☐ **M5-06** Demo design: write `docs/demo_script.md` (the 03 §6 storyline with timings, exact commands and expected screens) and the holdout-botnet moment. Agree with M2 on which family the holdout bundle leaves out. *Deps: M5-05, M1-09*
 - ☐ **M5-07** (*stretch*) Own-lab variant capture: two VMs on a host-only network, slow nmap scan / hping3 against **our own VM only**, fixed CICFlowMeter → CSV → replay. Document the safety and legality notes. *Deps: M5-05*
 - ☐ **M5-08** Fix the **Round 1 doc** using 02 A1–A10, and build the **pitch deck. Q&A prep sheet** covering: "it's supervised, how does it catch novel attacks?", "what if the model is wrong?", "dataset is lab traffic", "1% FPR at scale?", "drift?". *Deps: —, then update after I2*
 - ☐ **M5-09** Run 2 timed end-to-end dry runs and record the fallback video. Cross-check every deck claim against what's actually built. *Deps: I2*
-

Working agreements

- **Branches:** `M3/04-flows-endpoint` → PR to `main`. Small PRs, at least one reviewer, CI must pass.
 - **Contracts:** any change under `nscore/contracts/` needs approval from M3 plus one consumer, a bump to `CONTRACT_VERSION`, and regenerated fixtures (`python -m nscore.contracts.fixtures.make_fixtures`).
 - **Data and secrets:** never commit `data/`, `artifacts/` or `.env`. Gitleaks runs in CI.
 - **Honesty rule:** no number goes in the deck unless the repo reproduces it.
 - **Stuck for more than 2 hours on a dependency?** Build against the fixture or mock, and flag it in the track issue.
-

Source: `docs/04_tasks.md` · github.com/Saroy-ctrl/netsentinel